DevOps Laboratory

Assignment 4

Docker Containerization and Docker Compose

Student Name	YUVRAJ SHEMBALE
Roll No / PRN	123B1D045
Date of Submission	

1. Aim

To containerize a simple Python application using Docker, understand the difference between a Docker image and a container, and manage a multi-service application using Docker Compose.

2. Introduction

Docker is a containerization platform that packages an application together with all of its dependencies, libraries, and configuration into a single, portable unit called a container. Containers provide a consistent runtime environment, ensuring that an application behaves identically across development, testing, and production environments.

Docker Compose extends this capability by allowing multiple containers (services) to be defined and managed together through a single declarative YAML file. This is particularly valuable for modern microservices-based applications where several interdependent services (e.g., a web app, a database, and a cache) must be started, networked, and stopped as a coordinated unit.

3. Technologies / Tools Used

Technology	Purpose / Role
Docker Engine	Builds and runs containers from images
Docker CLI	Command-line client used to interact with Docker
Dockerfile	Text file containing instructions to build an application image
Docker Hub / Registry	Stores and distributes container images
Docker Compose	Tool to define and manage multi-container applications via YAML
Python 3.12	Runtime used for the sample containerized application

4. Project / Repository Structure

```
docker-python-demo/
■■■ app.py                # sample Python application
■■■ requirements.txt      # Python dependencies
■■■ Dockerfile            # instructions to build the image
■■■ .dockerignore         # files excluded from the build context
■■■ docker-compose.yml    # multi-container configuration
```

5. Procedure

1. Create a simple Python application (app.py) that performs a basic task and prints output.
2. Create a requirements.txt file listing any required Python dependencies.
3. Write a Dockerfile that specifies a base image, working directory, dependency installation, and startup command.
4. Build a Docker image from the Dockerfile using docker build -t .
5. Run the application inside a container using docker run, mapping ports as needed.
6. Verify the application output in the terminal or via a browser (for a web-based app).
7. View the container logs using docker logs to confirm correct execution.
8. Stop and remove the container using docker stop and docker rm.

9. Create a docker-compose.yml file describing the application (and any additional services) as a Compose configuration.

10. Use docker compose up and docker compose down to start and stop the full application stack.

6. Code / Configuration Snippets

6.1 Sample Python Application (app.py)

```
from flask import Flask

app = Flask(__name__)

@app.route('/')
def home():
    return 'Hello from a Dockerized Python application!'

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

6.2 Dependencies (requirements.txt)

```
flask==3.0.3
```

6.3 Dockerfile

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 5000
CMD ["python", "app.py"]
```

6.4 Building and Running the Container

```
$ docker build -t python-demo-app .
$ docker images
$ docker run -d -p 5000:5000 --name demo-container python-demo-app
$ docker ps
$ docker logs demo-container
$ docker stop demo-container
$ docker rm demo-container
```

6.5 Docker Compose Configuration (docker-compose.yml)

```
version: '3.9'

services:
  web:
    build: .
    ports:
      - "5000:5000"
    restart: unless-stopped
```

6.6 Docker Compose Commands

```
$ docker compose up -d
$ docker compose ps
$ docker compose logs -f
$ docker compose down
```

7. Output / Screenshots

The following screenshots demonstrate successful execution of each major step. Replace each placeholder with the corresponding screenshot captured during the practical.

```
PS D:\Devops Lab\Lab Assignments\Assignment 4\docker-demo> docker build -t python-demo-app .
[+] Building 30.0s (10/10) FINISHED
 => [internal] load build definition from Dockerfile
 => [internal] load metadata for docker.io/library/python:3.12-slim
 => [1/5] FROM docker.io/library/python:3.12-slim
 => [2/5] WORKDIR /app
 => [3/5] COPY requirements.txt .
 => [4/5] RUN pip install --no-cache-dir -r requirements.txt
 => [5/5] COPY . .
 => naming to docker.io/library/python-demo-app:latest done
```

Fig 1: Output of docker build showing the image build process

```
PS D:\Devops Lab\Lab Assignments\Assignment 4\docker-demo> docker images
IMAGE                  ID            DISK USAGE   CONTENT SIZE
python-demo-app:latest abe4c9bada66  198MB        48.3MB
python:3.12-slim       cb33f7f771ec  140MB        35.2MB
redis:7-alpine         858f009f9709  57.8MB       16.7MB
```

Fig 2: docker images listing the newly built image

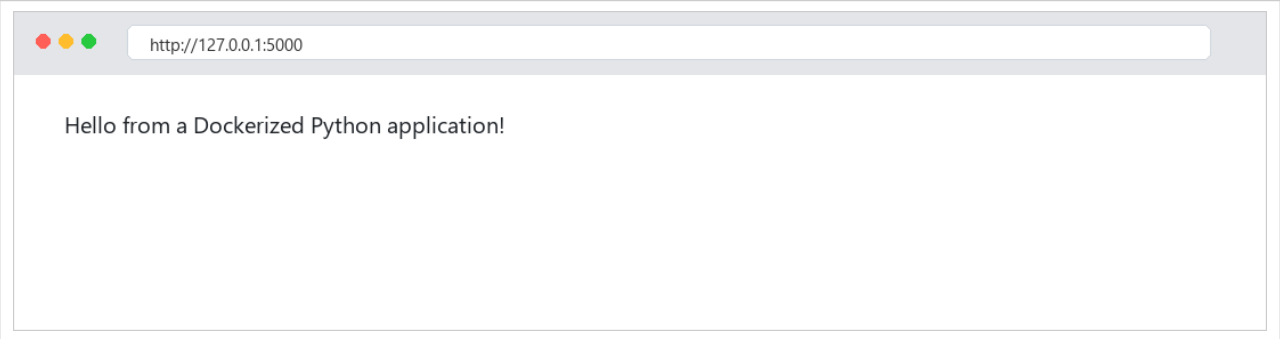


Fig 3: Application output verified in browser / terminal

```
PS D:\Devops Lab\Lab Assignments\Assignment 4\docker-demo> docker logs demo-container
 * Serving Flask app 'app'
 * Debug mode: off
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.17.0.4:5000
172.17.0.1 - - [22/Oct/2026 18:04:05] "GET / HTTP/1.1" 200 -
PS D:\Devops Lab\Lab Assignments\Assignment 4\docker-demo>
```

Fig 4: docker logs output for the running container

```
PS D:\Devops Lab\Lab Assignments\Assignment 4\docker-demo> docker compose up -d
[+] Running 2/2
 ✔ Network assignment-4-docker-python_default Created
 ✔ Container assignment-4-docker-python-web-1 Started
PS D:\Devops Lab\Lab Assignments\Assignment 4\docker-demo> curl http://localhost:5000
Hello from a Dockerized Python application!
PS D:\Devops Lab\Lab Assignments\Assignment 4\docker-demo>
```

Fig 5: docker compose up starting the application stack

8. Best Practices

- Use small, appropriate base images (e.g., -slim or -alpine variants) to reduce image size.
- Do not store passwords or secrets inside Dockerfiles or images — use environment variables or secret managers.
- Use a .dockerignore file to exclude unnecessary files (e.g., .git, __pycache__) from the build context.
- Give meaningful names and tags to images and containers for easier management.
- Remove unused containers and images after testing using docker system prune to reclaim disk space.

9. Conclusion

Docker provides a consistent and portable environment for application deployment by packaging code and dependencies into lightweight, isolated containers. In this assignment, a simple Python Flask application was containerized: a Docker image was built from a Dockerfile, run as a container, verified, and then cleanly stopped and removed.

Docker Compose was then used to define and manage the same application declaratively, demonstrating how multiple services can be started and stopped together — an approach that scales naturally to real-world, microservices-based systems.